



Введение в Python

Докладчик: Евграфов Михаил



Что такое Python?

Определение

**Python - это мультипарадигмальный
высокоуровневый язык программирования
общего назначения с динамической
строгой типизацией и автоматическим
управлением памятью**

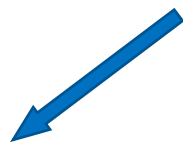


Определение

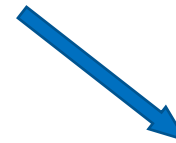
Python - это мультипарадигмальный высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью

Парадигмы программирования

Парадигмы программирования



императивная



декларативная

Императивный Python

```
total = 0
```

```
for number in range(1, 11):  
    if number % 2 == 0:  
        total += number
```

```
print(total)
```

```
# 30
```

OOP Python

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

account = BankAccount()
account.deposit(500)
print(account.balance)
# 500
```

Декларативный Python

```
from functools import reduce
```

```
prices = [100, 200, 50]
```

```
# Описываем преобразования, а не циклы:
```

```
total = reduce(lambda acc, x: acc + x, prices, 0)
```

```
cheap = list(filter(lambda p: p < 150, prices))
```

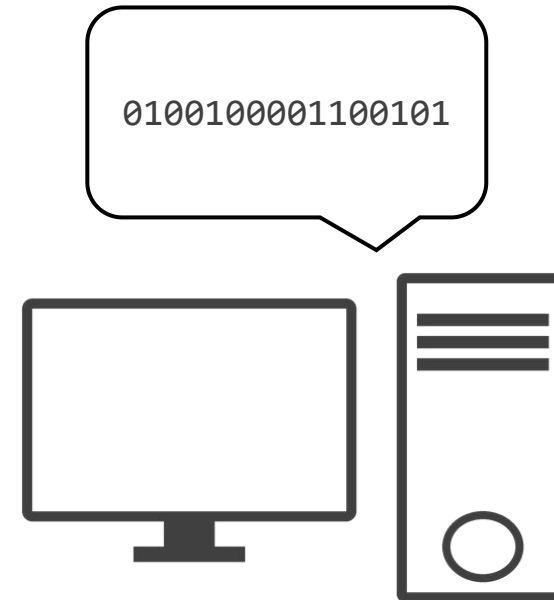
```
print(total)          # 350
```

```
print(cheap)          # [100, 50]
```

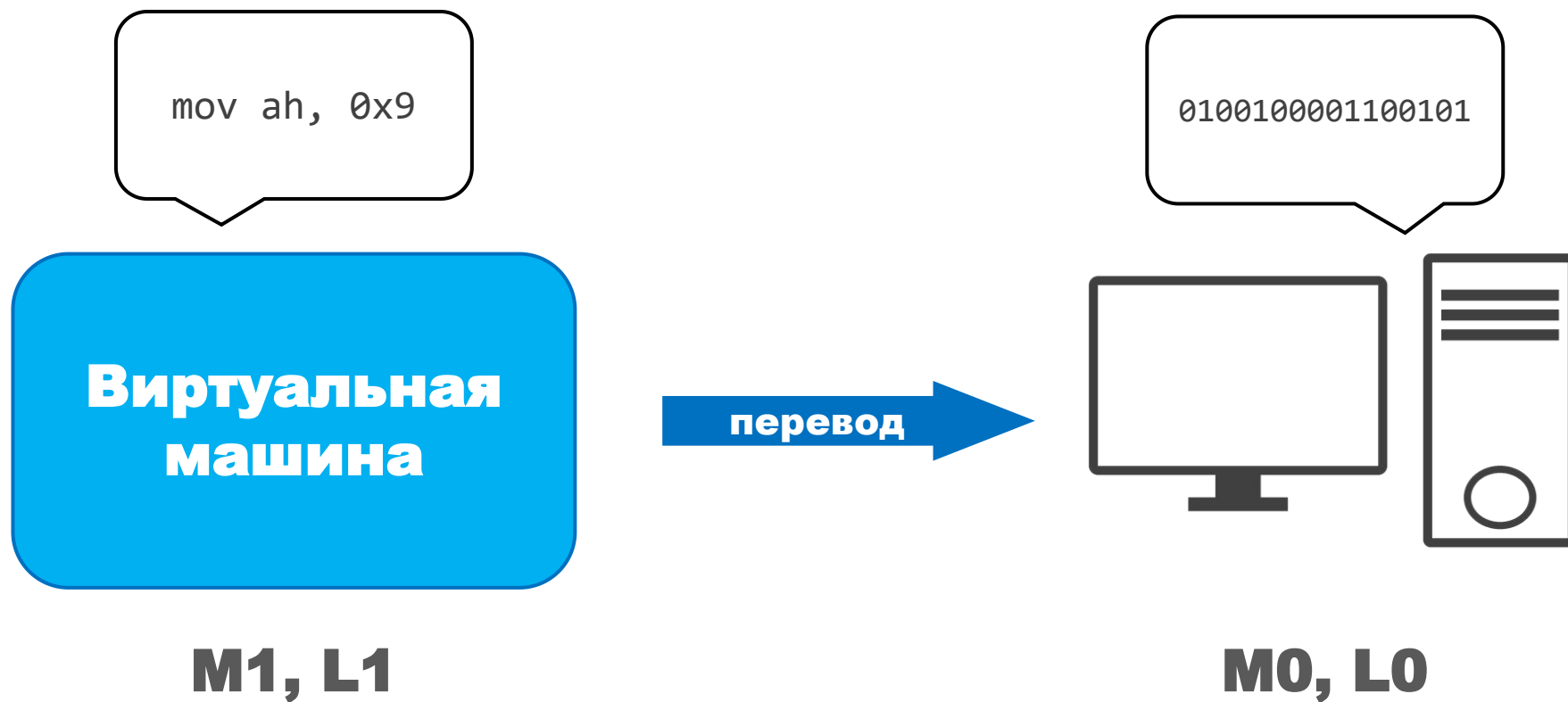

Определение

Python - это мультипарадигмальный **высокоуровневый** язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью

Люди и компьютеры



Виртуальные машины



Уровни абстракции



Пример различий

```
MOV AX, 2  
MOV BX, 3  
ADD AX, BX
```

**ассемблер,
низкий уровень**

```
result = 2 + 3
```

**то же самое
на Python**

Высокоуровневость Python

```
banned_elems = set()
```

```
if elem not in banned_elems:
```

```
    ...
```

Определение

Python - это мультипарадигмальный
высокоуровневый **язык программирования**
общего назначения с динамической
строгой типизацией и автоматическим
управлением памятью

Язык общего назначения

- Решает задачи из разных сфер
- DSL — узкая специализация:
 - SQL — запросы к БД
 - HTML/CSS — разметка
 - Regex — шаблоны текста
- Python - один язык для всего



Определение

Python - это мультипарадигмальный
высокоуровневый язык программирования
общего назначения с динамической
строгой типизацией и автоматическим
управлением памятью

Статическая типизация

```
int sum_two_nums(int lhs, int rhs) // нужны целые числа
{
    return lhs + rhs; // вернет целое число, как должна
}
```

```
int main() // нужно вернуть целое число
{
    sum_two_nums(42, -1); // передали целые числа - ок
    return 0; // вернет целое число - ок
}
```

Статическая типизация

```
def add_two_objects(lhs, rhs):  
    return lhs + rhs # проверим тип во время вызова  
  
if add_two_objects(42, -1) < 0:  
    add_two_objects("string", 42) # не вызовем - ок  
  
else:  
    add_two_objects("hello", " word")
```

Определение

Python - это мультипарадигмальный
высокоуровневый **язык программирования**
общего назначения с динамической
строгой типизацией и автоматическим
управлением памятью

Строгая и нестрогая типизация

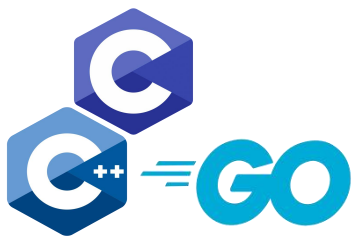
- **Нестрогая - частые неявные преобразования**

```
>>> "banana" + 42 # OK
```

- **Строгая - почти нет неявных преобразований**

```
>>> "banana" + 42 # Error
```

Градиенты типизации



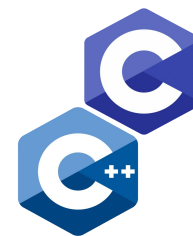
статическая



динамическая



строгая



нестрогая

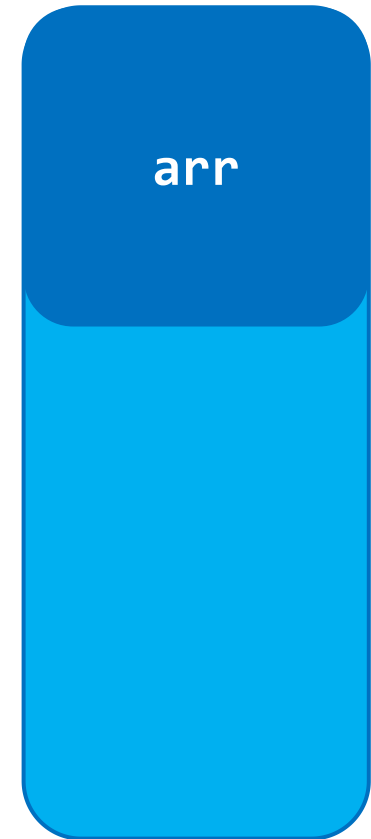
Определение

Python - это мультипарадигмальный
высокоуровневый **язык программирования**
общего назначения с динамической
строгой типизацией и **автоматическим**
управлением памятью

Ручное управление памятью

```
int *arr = malloc(100 * sizeof(int));  
...  
free(arr);
```

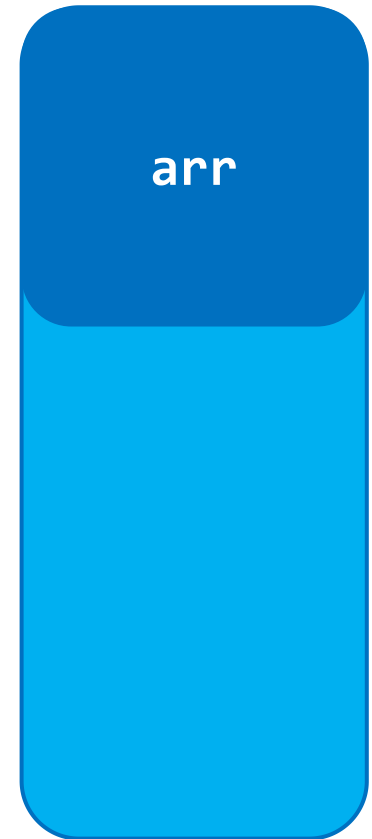
память



Утечки памяти

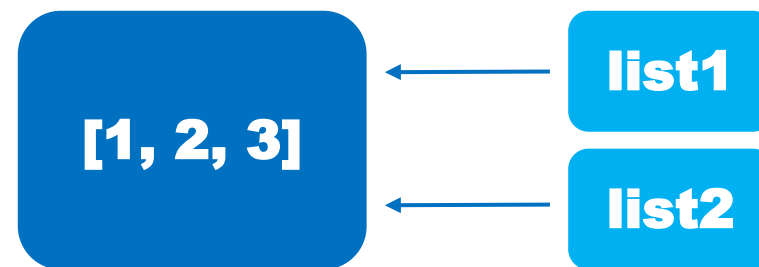
```
int *arr = malloc(100 * sizeof(int));  
...  
// free(arr); - забыли вызвать
```

память



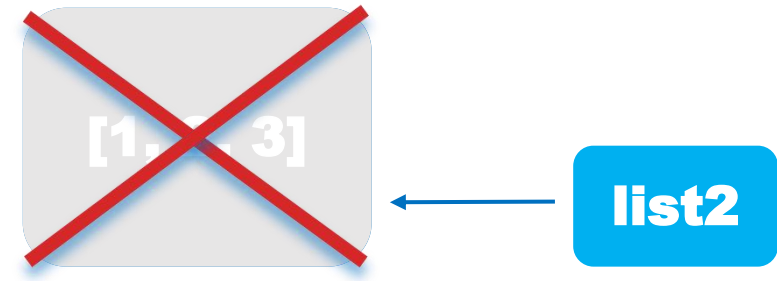
Подсчёт ссылок

```
list1 = [1, 2, 3]  
list2 = list1
```



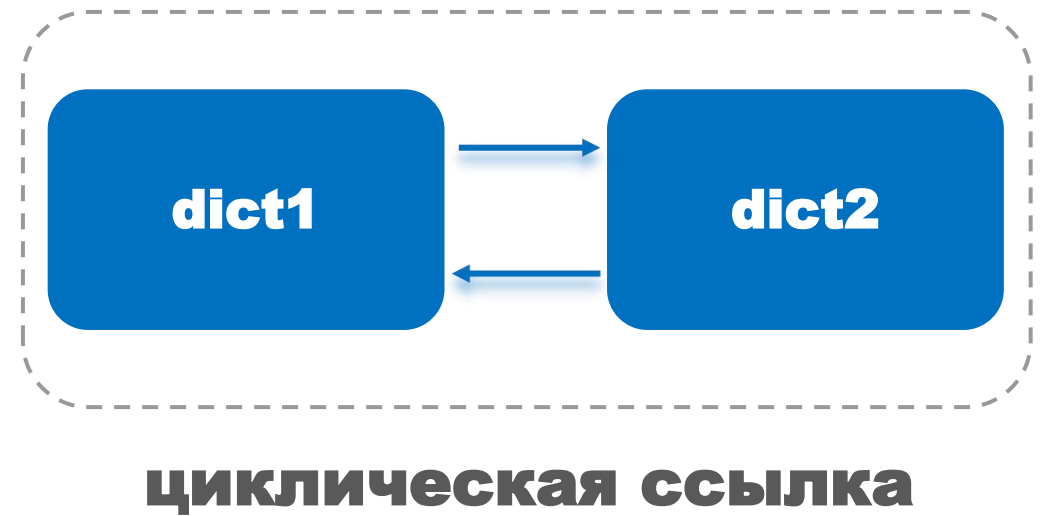
Подсчёт ссылок

```
list1 = [1, 2, 3]  
list2 = list1  
list1 = None  
list2 = None
```



Сборщик мусора для циклов

```
dict1 = {}  
dict2 = {}  
# циклические ссылки  
dict1["ref"] = dict2  
dict2["ref"] = dict1
```



Итог

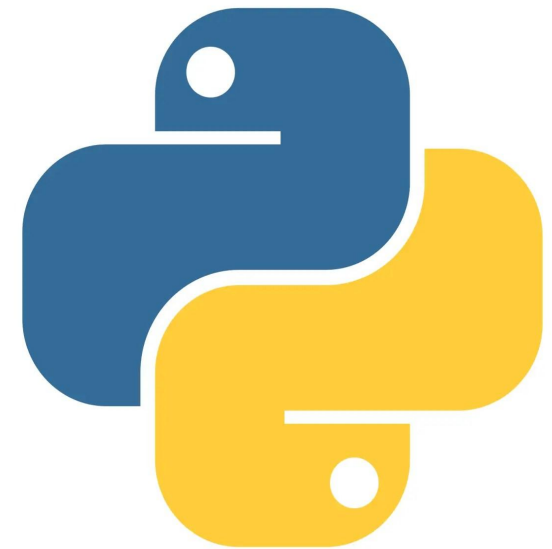
**Python - это язык программирования,
который подойдет для ваших задач**



Реализации Python

CPython

- Написана на C
- Байт-код + своя виртуальная машина
- Плюсы:
 - максимальная совместимость
 - огромная экосистема
 - стабильность
- Минусы:
 - медленнее «чистый» код
 - GIL — один поток одновременно
- Выбор по умолчанию — изучаем на курсе



PyPy

- Написана на RPython
- JIT: «горячий» код → машинный код
- Плюсы:
 - ускорение на долгих вычислениях
- Минусы:
 - медленный «прогрев»
 - слабая поддержка C-расширений
- Когда: долгие вычисления, важна скорость



Jython

- **Jython — Python на JVM**
- **Байт-код Java**
- **Плюсы:**
 - **доступ к Java-библиотекам**
 - **нет GIL**
- **Минусы:**
 - **развитие отстаёт**
 - **нет C-расширений**



IronPython

- **IronPython — Python на .NET**
- **Байт-код CLR**
- **Плюсы:**
 - **доступ к .NET и C#**
 - **нет GIL**
- **Минусы:**
 - **версии — с задержкой**
 - **слабая поддержка C-расширений**



Вывод

- **Одна спецификация, много реализаций**
- **CPython — стандарт по умолчанию**
- **PyPy — скорость; Jython / IronPython — интеграция с JVM и .NET**
- **Альтернатива — только если нужно её преимущество**



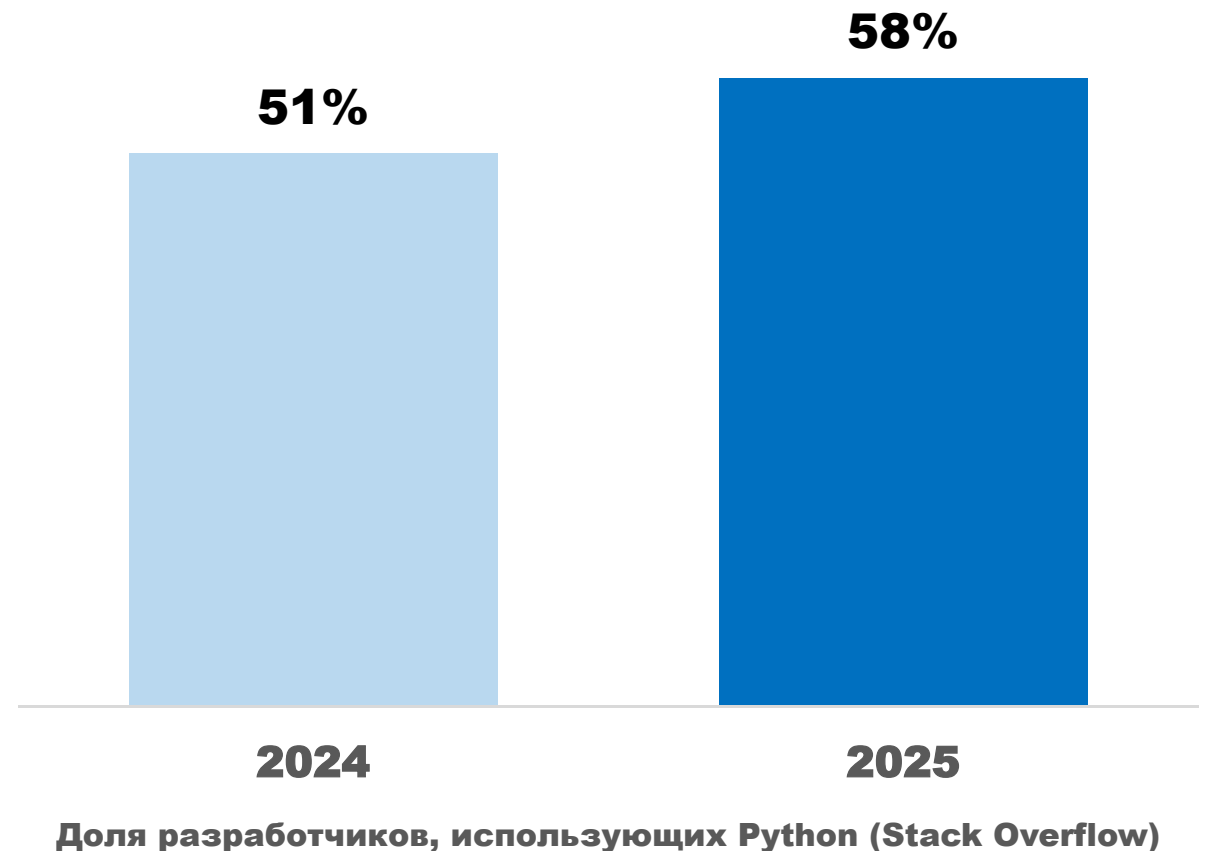
Зачем учить Python?

Где применяют Python сегодня

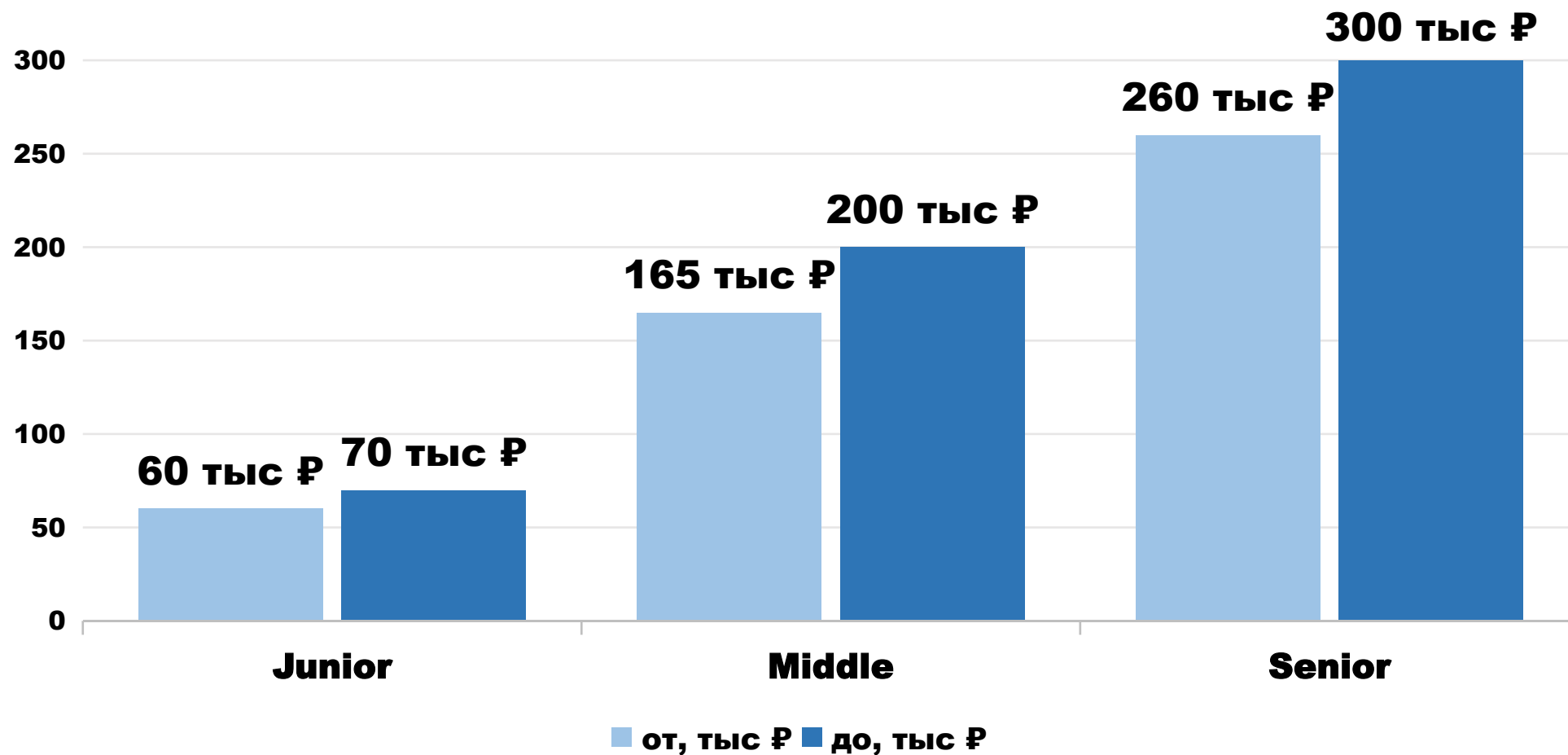
Сфера	Что делают	Инструменты
Data Science	анализ данных, дашборды	pandas, NumPy, Polars
ML и ИИ	модели, работа с LLM	PyTorch, scikit-learn, HF
Веб-бэкенд	серверы, API	Django, FastAPI, Flask
Автоматизация	файлы, парсинг сайтов	requests, BeautifulSoup
DevOps	CI/CD, облака	Ansible, boto3
Тестирование	автотесты	pytest, Selenium
Научные вычисления	физика, биоинформатика	SciPy, SymPy
Боты	Telegram, чат-боты	aiogram

Python — язык ИИ

- TensorFlow, PyTorch, scikit-learn — на Python
- GitHub Octoverse 2025: лидер в ИИ и данных
- Язык, который хотят изучить следующим



Зарплаты по грейдам



hh.ru Карьера, 2025 · медиана по Хабр Карьере — 234–243 тыс. ₺

Спрос на специалистов

- **Топ по вакансиям на hh.ru — наравне с Java, JS**
- **Растёт спрос: Python + ML + Data Engineering**
- **Спрос и зарплаты растут вместе с ИИ-агентами**

Учить язык или довериться ИИ?

- **ИИ хорошо умеет:**

- **типовой код быстрее человека**

- **объяснять и искать ошибки**

- **Реальное ускорение работы. Но...**

Зачем знать язык

- **Кто-то должен проверять результат**
- **Задачу нужно уметь поставить**
- **Отладка чужого кода — не «с нуля»**
- **Ответственность — на разработчике**
- **Рынок это подтверждает**

Как относиться к ИИ на курсе

- **На старте — код самому**
- **Дальше — ИИ как ускоритель**



Спасибо за внимание!